

데이터 엔지니어링 학습 현황 요약

목표: 데이터 수집부터 RAG 활용까지 연결되는 실습 환경 구축

uv 기반 가상환경, Python, LangChain, CSV/TXT 문서 로딩을 중심으로 학습 진행

01 개발 환경

uv를 활용해 프로젝트 단위 가상환경과 의존성을 관리하고, VS Code와 Jupyter Notebook에서 동일한 Python 환경을 사용하도록 구성합니다.

02 문서 데이터 처리

TXT, CSV, PDF와 같은 파일을 Document 객체로 불러오고 metadata와 page_content의 역할을 확인합니다.

03 RAG 기초

문서 로딩 이후 텍스트 분할, 임베딩, 벡터 저장소, 검색, LLM 답변 생성으로 이어지는 전체 흐름을 실습합니다.

04 협업과 형상관리

GitHub의 main, dev, 개인 브랜치 구조를 이해하고, 프로젝트 내부에 중첩 Git 저장소가 생기지 않도록 저장소 구조를 관리합니다.

핵심 요약

현재 단계는 '환경 구축 → 문서 로딩 → 데이터 구조 확인'에 해당합니다.

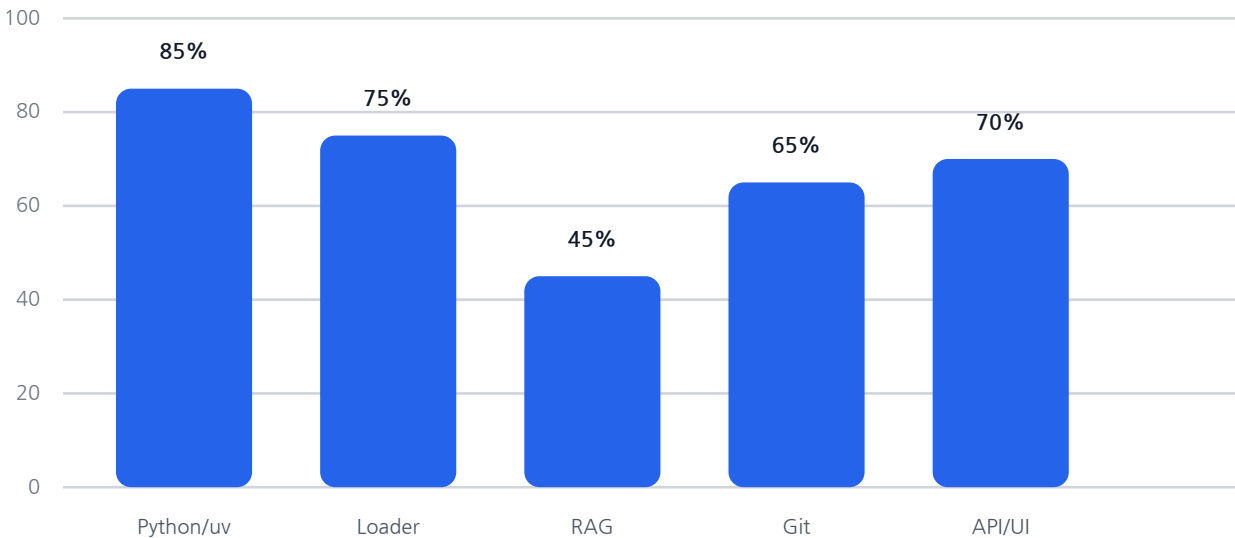
다음 단계에서는 문서 분할과 벡터 검색을 연결하여 실제 질의응답 흐름을 완성합니다.

학습 지표와 세부 분석

학습 영역별 진행 상태 - 예시 데이터

학습 영역	진행도	현재 상태	다음 실습
Python / uv	85%	환경 구성 완료	의존성 관리
LangChain Loader	75%	TXT/CSV 로딩	PDF Loader
RAG Pipeline	45%	구조 이해	Splitter + Embedding
Git / GitHub	65%	브랜치 개념	Merge / PR
FastAPI / Streamlit	70%	기초 연동	RAG API 연결

영역별 진행도 차트



해석

RAG 파이프라인 영역의 진척도가 상대적으로 낮으므로 다음 실습은 Splitter와 Embedding 연결에 집중합니다.

향후 학습 방향과 실행 로드맵

STEP 1

문서 로딩 안정화

TXT, CSV, PDF를 동일한 방식으로 불러오고 metadata, page_content, encoding 이슈를 정리합니다.

STEP 2

문서 분할과 임베딩

RecursiveCharacterTextSplitter를 활용해 chunk를 만들고 임베딩 벡터가 생성되는 과정을 확인합니다.

STEP 3

벡터 검색 연결

FAISS 또는 Chroma에 데이터를 저장하고 질문과 유사한 문서 조각을 검색하는 과정을 구현합니다.

STEP 4

RAG 서비스화

FastAPI 백엔드와 Streamlit UI를 연결해 파일 업로드, 검색, 답변 생성을 하나의 서비스로 구성합니다.

최종 목표

단순히 예제 코드를 실행하는 수준을 넘어,

1) 데이터를 읽고 2) 구조화하고 3) 검색하고 4) API와 UI로 제공하는

작은 데이터 서비스 한 개를 처음부터 끝까지 직접 구현하는 것을 목표로 합니다.